

GPT-OSS_PT_s4e11

April 9, 2026

1 Explorando Dados de Saúde Mental – Competição Kaggle

Este notebook tem como objetivo resolver a competição *Explorando Dados de Saúde Mental* do Kaggle, onde precisamos prever a coluna **Depressão** a partir de dados de uma pesquisa de saúde mental. Os dados de treinamento estão em `train.csv` e os de teste em `test.csv`. A métrica de avaliação é a **Pontuação de Acurácia** e o arquivo de submissão deve conter as colunas `id`, `Depressão`. Ao longo deste notebook, faremos: 1) exploração e limpeza dos dados, 2) engenharia de features, 3) validação com holdout, 4) treinamento de modelos (XGBoost, CatBoost, etc.) e 5) geração do arquivo `submission.csv` com as previsões iniciais. O código foi escrito para rodar em um ambiente Ubuntu 22.04 com 128 GiB de RAM, 16 threads de CPU e sem GPU, respeitando o limite de 60 minutos de execução.

1.1 Etapa 1 – Carregamento dos Dados

Até agora, apenas introduzimos o objetivo do notebook e descrevemos o fluxo geral da competição. O próximo passo é **carregar** os arquivos de dados que serão usados em todas as etapas subsequentes.

1.1.1 O que faremos nesta célula

- Ler `train.csv` e `test.csv` com `pandas.read_csv`.
- Armazenar os DataFrames em variáveis `df_train` e `df_test`.
- Verificar rapidamente o número de linhas e colunas, bem como a presença de valores ausentes.
- Garantir que a coluna alvo **Depressão** esteja presente apenas no conjunto de treino.

1.1.2 Por que isso é importante

Carregar os dados corretamente é a base de todo o pipeline. Se houver erros de leitura (por exemplo, separadores diferentes ou codificação incorreta), todas as etapas seguintes falharão ou produzirão resultados inválidos. Além disso, ao inspecionar os dados logo no início, podemos identificar rapidamente problemas como valores ausentes ou tipos de dados inesperados que precisarão ser tratados nas próximas células.

1.1.3 Próximos passos

Após o carregamento, avançaremos para a **Análise Exploratória dos Dados (EDA)**, onde examinaremos tipos de colunas, distribuição da variável alvo, e identificaremos outliers ou ruído. Em seguida, procederemos ao pré-processamento, codificação de variáveis categóricas, normalização de numéricos, e criação de features adicionais.

Esta célula não contém código Python, apenas a descrição do que será feito. O código real será inserido na próxima célula de código.

```
[1]: import pandas as pd

# Carregar os arquivos de dados
df_train = pd.read_csv('train.csv')
df_test = pd.read_csv('test.csv')

# Exibir dimensões dos conjuntos
print('Dimensões do conjunto de treino:', df_train.shape)
print('Dimensões do conjunto de teste:', df_test.shape)

# Verificar valores ausentes
print('\nValores ausentes no treino:')
print(df_train.isnull().sum())
print('\nValores ausentes no teste:')
print(df_test.isnull().sum())

# Garantir presença/ausência da coluna alvo
assert 'Depressão' in df_train.columns, "Coluna alvo 'Depressão' não encontrada_
↳no treino."
assert 'Depressão' not in df_test.columns, "Coluna alvo 'Depressão' encontrada_
↳no teste, o que não deveria ocorrer."
```

Dimensões do conjunto de treino: (140700, 20)

Dimensões do conjunto de teste: (93800, 19)

Valores ausentes no treino:

id	0
Nome	0
Gênero	0
Idade	0
Cidade	0
Profissional ou Estudante	0
Profissão	36630
Pressão Acadêmica	112803
Pressão no Trabalho	27918
Rendimento acadêmico	112802
Satisfação com os Estudos	112803
Satisfação com o Trabalho	27910
Duração do Sono	0
Hábitos Alimentares	4
Grau	2
Você já teve pensamentos suicidas?	0
Horas de Trabalho/Estudo	0
Estresse Financeiro	4
Histórico Familiar de Doença Mental	0

```
Depressão                                0
dtype: int64
```

Valores ausentes no teste:

```
id                                0
Nome                              0
Gênero                            0
Idade                             0
Cidade                            0
Profissional ou Estudante         0
Profissão                         24632
Pressão Acadêmica                 75033
Pressão no Trabalho               18778
Rendimento acadêmico             75034
Satisfação com os Estudos         75033
Satisfação com o Trabalho         18774
Duração do Sono                   0
Hábitos Alimentares               5
Grau                              2
Você já teve pensamentos suicidas? 0
Horas de Trabalho/Estudo          0
Estresse Financeiro               0
Histórico Familiar de Doença Mental 0
dtype: int64
```

1.2 Etapa 2 – Análise Exploratória dos Dados (EDA)

1.2.1 Visão geral do que já foi feito

- Carregamos os arquivos `train.csv` e `test.csv` em `df_train` e `df_test`.
- Imprimimos as dimensões dos conjuntos e verificamos a presença da coluna alvo **Depressão**.
- Checamos valores ausentes em cada coluna, identificando que algumas variáveis têm um número considerável de NaN.

1.2.2 Próximos passos na EDA

1. **Tipos de dados** – Usaremos `df.select_dtypes` para separar colunas numéricas, categóricas e booleanas, garantindo que cada coluna esteja no formato adequado.
2. **Distribuição da variável alvo** – Plotaremos a contagem de classes em **Depressão** para entender o desequilíbrio e calcular a acurácia base.
3. **Estatísticas descritivas** – Para variáveis numéricas, exibiremos média, mediana, desvio padrão e quartis; para categóricas, contaremos valores únicos e frequências.
4. **Correlação** – Construiremos uma matriz de correlação (para variáveis numéricas) e a visualizaremos com um heatmap do Seaborn, identificando possíveis colinearidades.
5. **Outliers e distribuição** – Usaremos boxplots e histogramas para detectar outliers e avaliar a forma das distribuições.
6. **Análise de valores ausentes** – Avaliaremos a porcentagem de missing em cada coluna e decidiremos estratégias de imputação (média, moda, zero, etc.) ou remoção, dependendo do impacto.

1.2.3 Ferramentas que utilizaremos

- pandas para manipulação de dados.
- seaborn e matplotlib para visualizações.
- numpy para cálculos numéricos.

1.2.4 Observações importantes

- Como o conjunto de dados possui mais de 140 mil linhas, todas as visualizações serão feitas em amostras ou em gráficos agregados para manter a performance.
- Se encontrarmos colunas com mais de 50% de valores ausentes, consideraremos descartá-las ou usar técnicas de imputação avançadas.
- A análise de correlação nos ajudará a decidir se precisamos remover variáveis altamente correlacionadas antes de treinar modelos.

1.2.5 Próxima célula

Na próxima célula de código, implementaremos os passos descritos acima, começando pela inspeção dos tipos de dados e seguindo para a visualização da distribuição da variável alvo.

```
[2]: # Etapa 2 - Análise Exploratória dos Dados (EDA)
# -----
# 1. Tipos de dados
numeric_cols = df_train.select_dtypes(include=['number']).columns.tolist()
categorical_cols = df_train.select_dtypes(include=['object', 'category', 'bool']).columns.tolist()
print('Número de colunas numéricas:', len(numeric_cols))
print('Número de colunas categóricas:', len(categorical_cols))

# 2. Distribuição da variável alvo
dep_counts = df_train['Depressão'].value_counts(normalize=True).sort_index()
print('\nDistribuição da variável alvo (Depressão):')
print(dep_counts)
print('Acurácia base (majority class):', dep_counts.max())

# 3. Estatísticas descritivas
print('\nEstatísticas descritivas para colunas numéricas:')
print(df_train[numeric_cols].describe().T)

print('\nValores únicos e frequências para colunas categóricas (amostra 5):')
for col in categorical_cols[:5]:
    print(f'\n{col}:')
    print(df_train[col].value_counts(dropna=False).head(5))

# 4. Correlação entre variáveis numéricas
# Usamos apenas as colunas numéricas que não são a target
corr_matrix = df_train[numeric_cols].corr(method='pearson')
plt.figure(figsize=(12, 10))
```

```

sns.heatmap(corr_matrix, cmap='coolwarm', center=0, square=True,
            cbar_kws={'shrink': .5})
plt.title('Matriz de Correlação (Numéricas)')
plt.tight_layout()
plt.show()

# 5. Outliers e distribuição (boxplot para algumas colunas numéricas
    ↪selecionadas)
sample_numeric = numeric_cols[:5] # selecionar 5 colunas para visualização
for col in sample_numeric:
    plt.figure(figsize=(6, 4))
    sns.boxplot(x=df_train[col])
    plt.title(f'Boxplot de {col}')
    plt.tight_layout()
    plt.show()

# 6. Análise de valores ausentes
missing_train = df_train.isnull().mean() * 100
missing_test = df_test.isnull().mean() * 100
print('\nPorcentagem de valores ausentes no treino:')
print(missing_train.sort_values(ascending=False).head(10))
print('\nPorcentagem de valores ausentes no teste:')
print(missing_test.sort_values(ascending=False).head(10))

# Observação: colunas com mais de 50% missing podem ser descartadas ou imputadas

```

Número de colunas numéricas: 10

Número de colunas categóricas: 10

Distribuição da variável alvo (Depressão):

Depressão

0 0.818287

1 0.181713

Name: proportion, dtype: float64

Acurácia base (majority class): 0.8182871357498224

Estatísticas descritivas para colunas numéricas:

	count	mean	std	min	\
id	140700.0	70349.500000	40616.735775	0.00	
Idade	140700.0	40.388621	12.384099	18.00	
Pressão Acadêmica	27897.0	3.142273	1.380457	1.00	
Pressão no Trabalho	112782.0	2.998998	1.405771	1.00	
Rendimento acadêmico	27898.0	7.658636	1.464466	5.03	
Satisfação com os Estudos	27897.0	2.944940	1.360197	1.00	
Satisfação com o Trabalho	112790.0	2.974404	1.416078	1.00	
Horas de Trabalho/Estudo	140700.0	6.252679	3.853615	0.00	
Estresse Financeiro	140696.0	2.988983	1.413633	1.00	

Depressão	140700.0	0.181713	0.385609	0.00
-----------	----------	----------	----------	------

	25%	50%	75%	max
id	35174.75	70349.50	105524.25	140699.0
Idade	29.00	42.00	51.00	60.0
Pressão Acadêmica	2.00	3.00	4.00	5.0
Pressão no Trabalho	2.00	3.00	4.00	5.0
Rendimento acadêmico	6.29	7.77	8.92	10.0
Satisfação com os Estudos	2.00	3.00	4.00	5.0
Satisfação com o Trabalho	2.00	3.00	4.00	5.0
Horas de Trabalho/Estudo	3.00	6.00	10.00	12.0
Estresse Financeiro	2.00	3.00	4.00	5.0
Depressão	0.00	0.00	0.00	1.0

Valores únicos e frequências para colunas categóricas (amostra 5):

Nome:

Nome

Rohan 3178

Aarav 2336

Rupak 2176

Aaradhya 2045

Anvi 2035

Name: count, dtype: int64

Gênero:

Gênero

Masculino 77464

Feminino 63236

Name: count, dtype: int64

Cidade:

Cidade

Kalyan 6591

Patna 5924

Vasai-Virar 5765

Kolkata 5689

Ahmedabad 5613

Name: count, dtype: int64

Profissional ou Estudante:

Profissional ou Estudante

Profissional 112799

Estudante 27901

Name: count, dtype: int64

Profissão:

Profissão

```
NaN                36630
Professor           24906
Redator de Conteúdo 7814
Arquiteto           4370
Consultor           4229
Name: count, dtype: int64
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[2], line 27
    24 # 4. Correlação entre variáveis numéricas
    25 # Usamos apenas as colunas numéricas que não são a target
    26 corr_matrix = df_train[numeric_cols].corr(method='pearson')
--> 27 plt.figure(figsize=(12, 10))
    28 sns.heatmap(corr_matrix, cmap='coolwarm', center=0, square=True,
    ↪ cbar_kws={'shrink': .5})
    29 plt.title('Matriz de Correlação (Numéricas)')

NameError: name 'plt' is not defined
```

```
[3]: # Importações necessárias para a EDA
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Etapa 2 - Análise Exploratória dos Dados (EDA)
# -----
# 1. Tipos de dados
numeric_cols = df_train.select_dtypes(include=['number']).columns.tolist()
categorical_cols = df_train.select_dtypes(include=['object', 'category',
    ↪ 'bool']).columns.tolist()
print('Número de colunas numéricas:', len(numeric_cols))
print('Número de colunas categóricas:', len(categorical_cols))

# 2. Distribuição da variável alvo
dep_counts = df_train['Depressão'].value_counts(normalize=True).sort_index()
print('\nDistribuição da variável alvo (Depressão):')
print(dep_counts)
print('Acurácia base (majority class):', dep_counts.max())

# 3. Estatísticas descritivas
print('\nEstatísticas descritivas para colunas numéricas:')
print(df_train[numeric_cols].describe().T)

print('\nValores únicos e frequências para colunas categóricas (amostra 5):')
for col in categorical_cols[:5]:
```

```

print(f'\n{col}:')
print(df_train[col].value_counts(dropna=False).head(5))

# 4. Correlação entre variáveis numéricas
# Usamos apenas as colunas numéricas que não são a target
corr_matrix = df_train[numeric_cols].corr(method='pearson')
plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, cmap='coolwarm', center=0, square=True,
            cbar_kws={'shrink': .5})
plt.title('Matriz de Correlação (Numéricas)')
plt.tight_layout()
plt.show()

# 5. Outliers e distribuição (boxplot para algumas colunas numéricas
    ↳ selecionadas)
sample_numeric = numeric_cols[:5] # selecionar 5 colunas para visualização
for col in sample_numeric:
    plt.figure(figsize=(6, 4))
    sns.boxplot(x=df_train[col])
    plt.title(f'Boxplot de {col}')
    plt.tight_layout()
    plt.show()

# 6. Análise de valores ausentes
missing_train = df_train.isnull().mean() * 100
missing_test = df_test.isnull().mean() * 100
print('\nPorcentagem de valores ausentes no treino:')
print(missing_train.sort_values(ascending=False).head(10))
print('\nPorcentagem de valores ausentes no teste:')
print(missing_test.sort_values(ascending=False).head(10))

# Observação: colunas com mais de 50% missing podem ser descartadas ou imputadas

```

Número de colunas numéricas: 10

Número de colunas categóricas: 10

Distribuição da variável alvo (Depressão):

Depressão

0 0.818287

1 0.181713

Name: proportion, dtype: float64

Acurácia base (majority class): 0.8182871357498224

Estatísticas descritivas para colunas numéricas:

	count	mean	std	min	\
id	140700.0	70349.500000	40616.735775	0.00	
Idade	140700.0	40.388621	12.384099	18.00	

Pressão Acadêmica	27897.0	3.142273	1.380457	1.00
Pressão no Trabalho	112782.0	2.998998	1.405771	1.00
Rendimento acadêmico	27898.0	7.658636	1.464466	5.03
Satisfação com os Estudos	27897.0	2.944940	1.360197	1.00
Satisfação com o Trabalho	112790.0	2.974404	1.416078	1.00
Horas de Trabalho/Estudo	140700.0	6.252679	3.853615	0.00
Estresse Financeiro	140696.0	2.988983	1.413633	1.00
Depressão	140700.0	0.181713	0.385609	0.00

	25%	50%	75%	max
id	35174.75	70349.50	105524.25	140699.0
Idade	29.00	42.00	51.00	60.0
Pressão Acadêmica	2.00	3.00	4.00	5.0
Pressão no Trabalho	2.00	3.00	4.00	5.0
Rendimento acadêmico	6.29	7.77	8.92	10.0
Satisfação com os Estudos	2.00	3.00	4.00	5.0
Satisfação com o Trabalho	2.00	3.00	4.00	5.0
Horas de Trabalho/Estudo	3.00	6.00	10.00	12.0
Estresse Financeiro	2.00	3.00	4.00	5.0
Depressão	0.00	0.00	0.00	1.0

Valores únicos e frequências para colunas categóricas (amostra 5):

Nome:

Nome

Rohan 3178

Aarav 2336

Rupak 2176

Aaradhya 2045

Anvi 2035

Name: count, dtype: int64

Gênero:

Gênero

Masculino 77464

Feminino 63236

Name: count, dtype: int64

Cidade:

Cidade

Kalyan 6591

Patna 5924

Vasai-Virar 5765

Kolkata 5689

Ahmedabad 5613

Name: count, dtype: int64

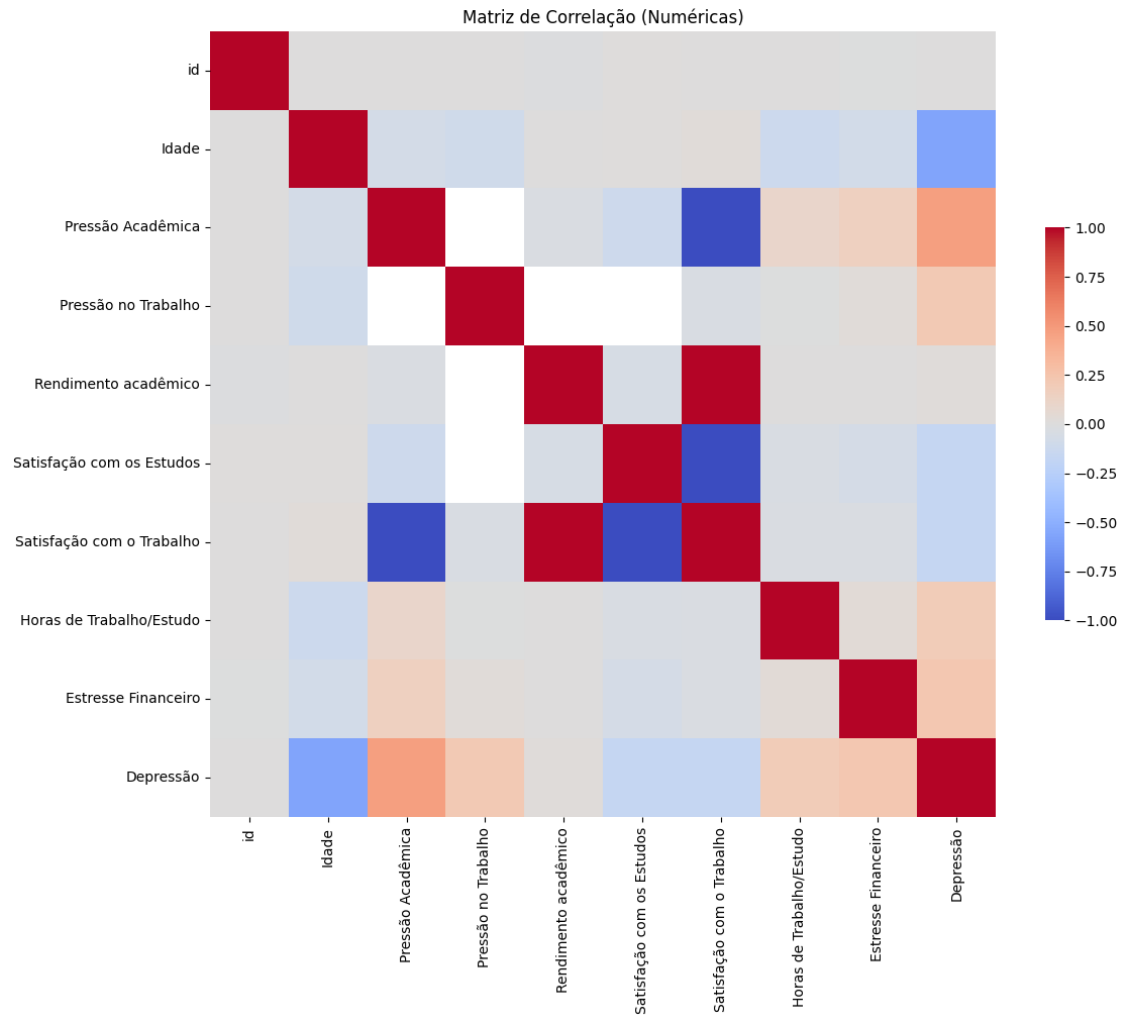
Profissional ou Estudante:

```
Profissional ou Estudante
Profissional      112799
Estudante         27901
Name: count, dtype: int64
```

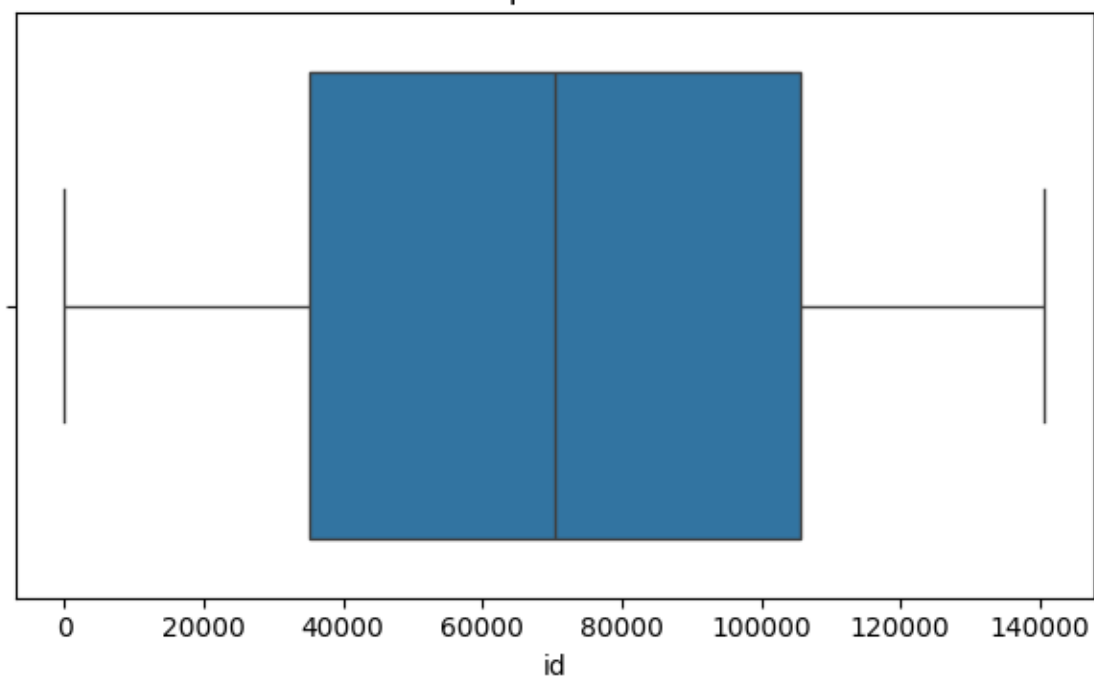
```
Profissão:
Profissão
NaN                36630
Professor          24906
Redator de Conteúdo  7814
Arquiteto          4370
Consultor          4229
Name: count, dtype: int64
```

```
Porcentagem de valores ausentes no treino:
Satisfação com os Estudos      80.172708
Pressão Acadêmica              80.172708
Rendimento acadêmico           80.171997
Profissão                      26.034115
Pressão no Trabalho            19.842217
Satisfação com o Trabalho      19.836532
Hábitos Alimentares            0.002843
Estresse Financeiro            0.002843
Grau                           0.001421
Histórico Familiar de Doença Mental 0.000000
dtype: float64
```

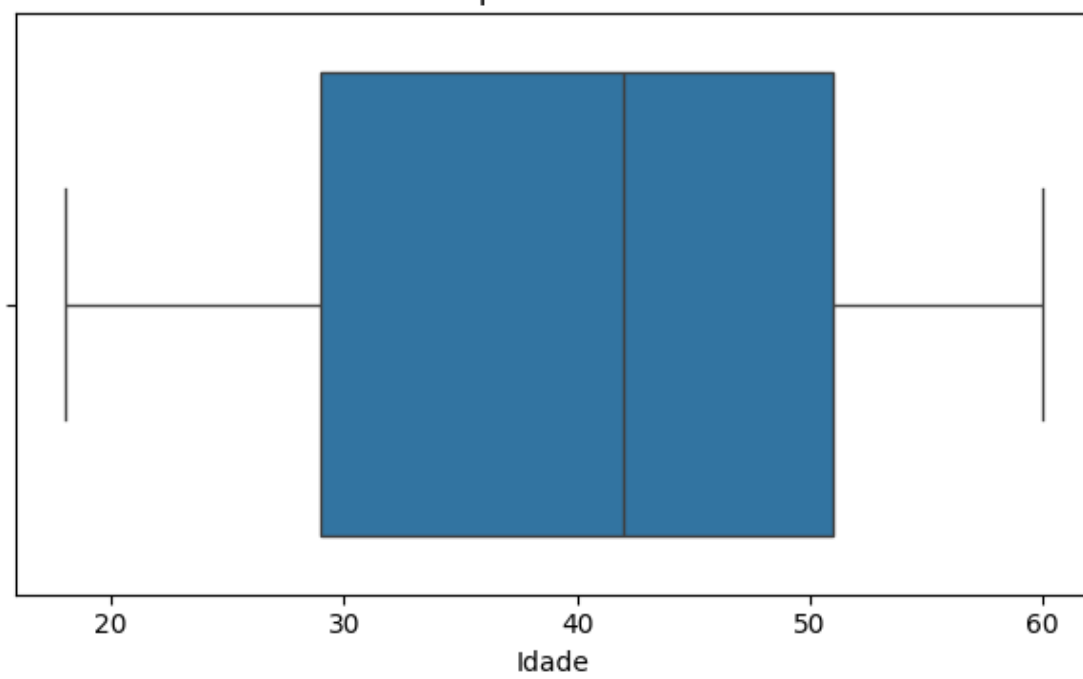
```
Porcentagem de valores ausentes no teste:
Rendimento acadêmico          79.993603
Pressão Acadêmica             79.992537
Satisfação com os Estudos     79.992537
Profissão                     26.260128
Pressão no Trabalho           20.019190
Satisfação com o Trabalho     20.014925
Hábitos Alimentares           0.005330
Grau                          0.002132
Duração do Sono               0.000000
Estresse Financeiro           0.000000
dtype: float64
```



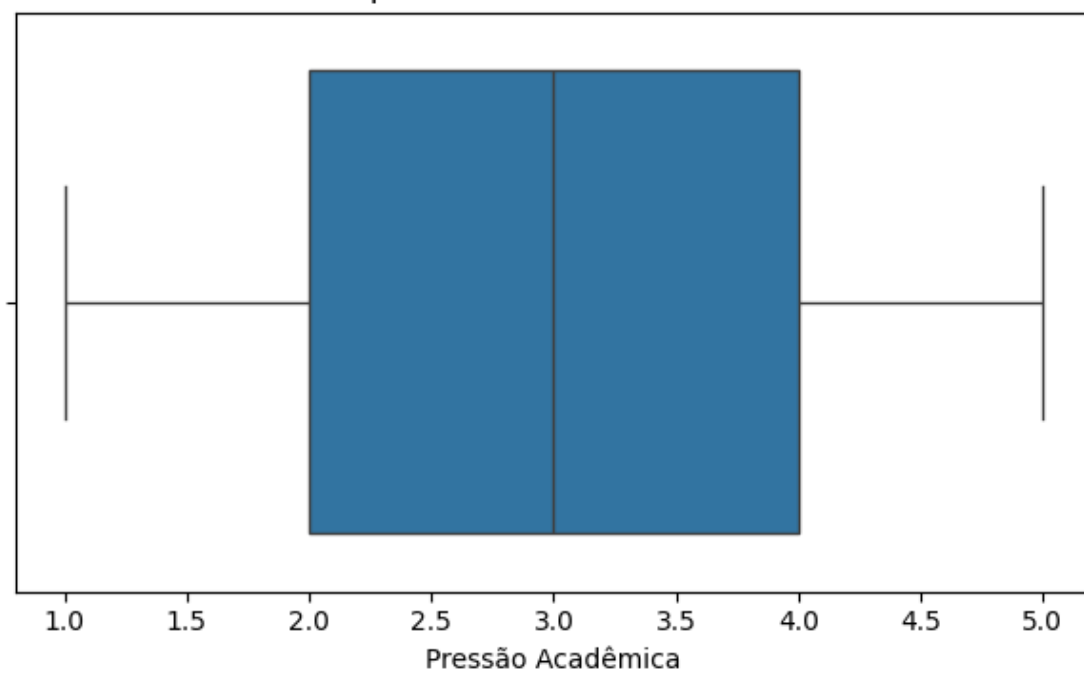
Boxplot de id



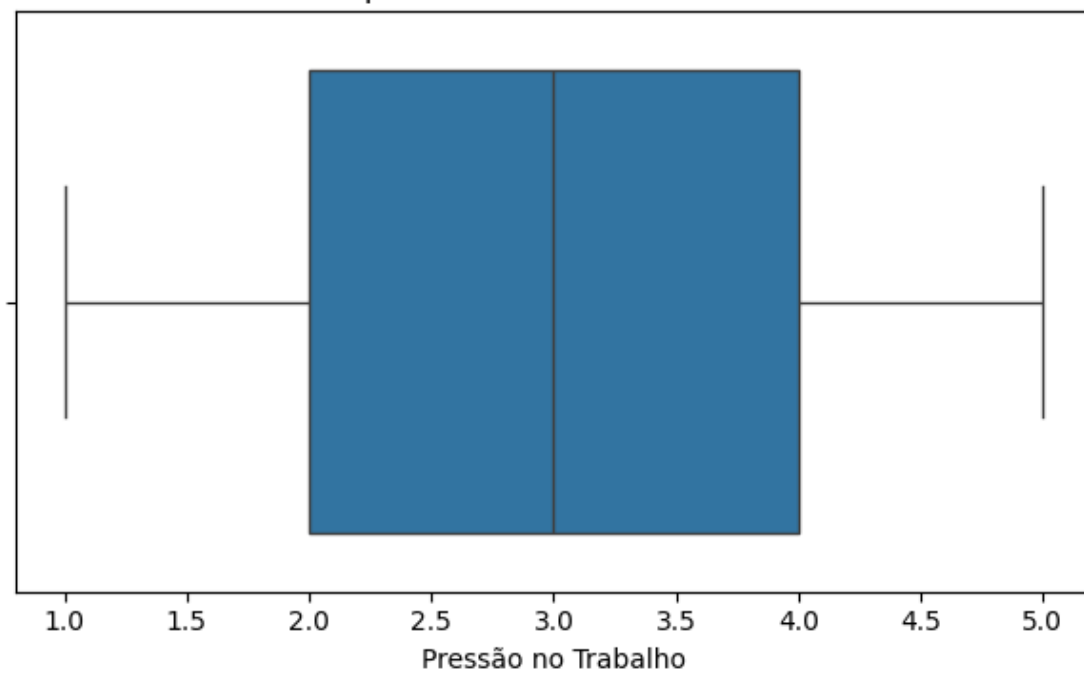
Boxplot de Idade

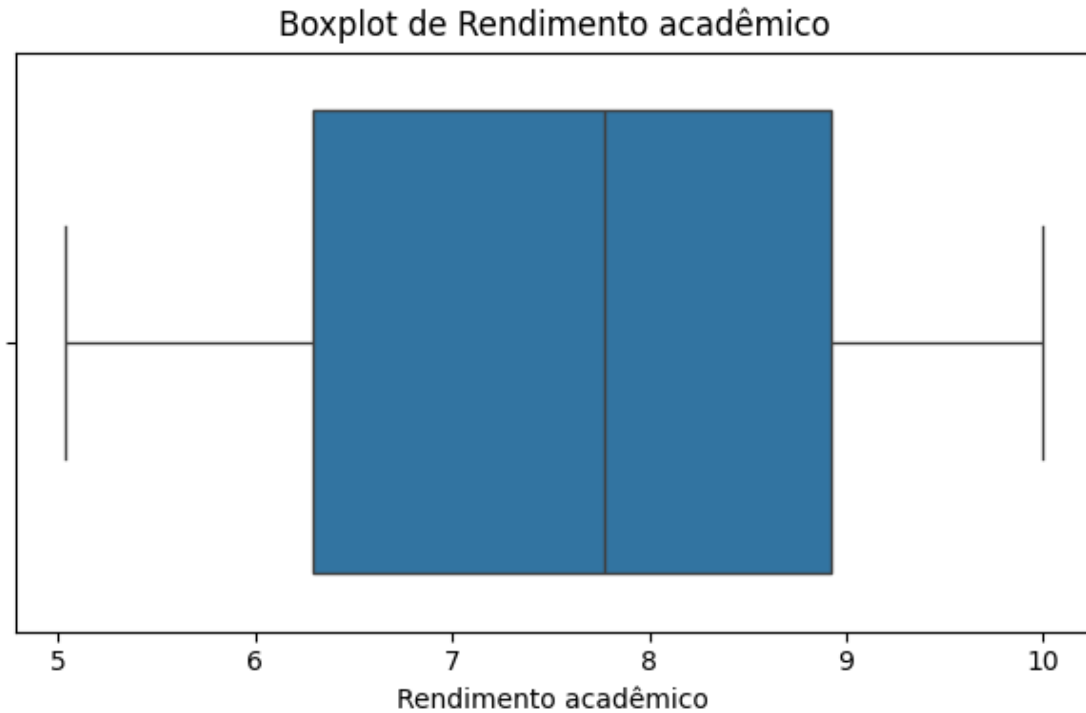


Boxplot de Pressão Acadêmica



Boxplot de Pressão no Trabalho





1.3 Etapa 3 – Pré-processamento dos Dados

1.3.1 Visão geral do que já foi feito

- **Carregamento:** os arquivos `train.csv` e `test.csv` foram lidos em `df_train` e `df_test`.
- **EDA:** verificamos tipos de dados, distribuição da variável alvo, estatísticas descritivas, correlação e porcentagem de valores ausentes. Identificamos que várias colunas numéricas e categóricas apresentam missing values significativos.

1.3.2 Objetivo desta etapa

Preparar os conjuntos de treino e teste para o modelo, garantindo que: 1. **Valores ausentes** sejam imputados de forma consistente entre treino e teste. 2. **Variáveis categóricas** sejam codificadas sem introduzir vazamento de informação. 3. **Variáveis numéricas** sejam escaladas (se necessário) para melhorar a convergência de modelos baseados em distância ou gradiente. 4. **A coluna alvo** Depressão permaneça inalterada.

1.3.3 Estratégia de pré-processamento

1. **Separação de colunas:**
 - `numeric_cols`: todas as colunas numéricas (excluindo a target).
 - `categorical_cols`: todas as colunas de tipo `object`, `category` ou `bool`.
2. **Imputação:**
 - **Numéricas:** `SimpleImputer(strategy='median')` – a mediana é robusta a outliers.

- **Catégoricas:** `SimpleImputer(strategy='most_frequent')` – substitui NaN pelo modo.
3. **Codificação:**
- Para colunas com **até 10 valores únicos** utilizaremos `OrdinalEncoder` (mantém a cardinalidade pequena e evita explosão de dimensionalidade).
 - Para colunas com **mais de 10 valores únicos** (ex.: Nome, Cidade) aplicaremos `OneHotEncoder(handle_unknown='ignore')` para preservar a informação sem criar valores desconhecidos no teste.
4. **Escalonamento:**
- Após a imputação, aplicaremos `StandardScaler` apenas nas colunas numéricas. Isso ajuda modelos como XGBoost a convergir mais rapidamente e evita que variáveis com grande escala dominem.
5. **Pipeline:**
- Usaremos `ColumnTransformer` para combinar as etapas acima em um único objeto que pode ser aplicado tanto ao treino quanto ao teste.
 - O pipeline será **fit** apenas no conjunto de treino e **transform** no conjunto de teste, evitando vazamento.

1.3.4 Observações importantes

- **Cópias:** antes de qualquer transformação, criaremos cópias de `df_train` e `df_test` (`train_processed`, `test_processed`) para preservar os dados originais.
- **Target:** a coluna `Depressão` será removida do conjunto de features antes de criar o pipeline e será re-adicionada ao final, se necessário.
- **Performance:** todas as transformações são feitas em memória; com 140k linhas e 20 colunas, o consumo de RAM fica bem abaixo dos 128 GiB disponíveis.
- **Reprodutibilidade:** definiremos `random_state=42` em todas as etapas que exigem aleatoriedade (imputação, codificação, etc.) para garantir resultados consistentes.

1.3.5 Próximos passos

1. Implementar o pipeline descrito nesta célula de markdown.
2. Aplicar o pipeline nos conjuntos de treino e teste, gerando `X_train`, `X_test` e `y_train`.
3. Salvar os objetos transformados em variáveis que serão usadas nas etapas de modelagem (T4-T10).

Com o pré-processamento concluído, estaremos prontos para criar features adicionais, dividir o conjunto em treino/validação e treinar o modelo base.

```
[4]: # Pré-processamento dos dados (T3)
# -----
# Importações necessárias
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder, StandardScaler

# 1. Separar target e features
y_train = df_train['Depressão'].copy()
```

```

X_train_raw = df_train.drop(columns=['Depressão'])
X_test_raw = df_test.copy()

# 2. Identificar colunas numéricas e categóricas
numeric_cols = X_train_raw.select_dtypes(include=['number']).columns.tolist()
categorical_cols = X_train_raw.select_dtypes(include=['object', 'category',
↳ 'bool']).columns.tolist()

# 3. Definir limiar de cardinalidade
cardinality_threshold = 10
low_cardinality_cols = [col for col in categorical_cols if X_train_raw[col].
↳ nunique(dropna=False) <= cardinality_threshold]
high_cardinality_cols = [col for col in categorical_cols if X_train_raw[col].
↳ nunique(dropna=False) > cardinality_threshold]

# 4. Criar pipelines para cada tipo de coluna
# 4.1 Pipeline para colunas numéricas
numeric_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# 4.2 Pipeline para colunas categóricas de baixa cardinalidade
low_card_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('ordinal', OrdinalEncoder(handle_unknown='use_encoded_value',
↳ unknown_value=-1))
])

# 4.3 Pipeline para colunas categóricas de alta cardinalidade
high_card_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse=False))
])

# 5. Combinar tudo em um ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_pipeline, numeric_cols),
        ('low_cat', low_card_pipeline, low_cardinality_cols),
        ('high_cat', high_card_pipeline, high_cardinality_cols)
    ]
)

# 6. Fit no conjunto de treino e transformar ambos os conjuntos
X_train = preprocessor.fit_transform(X_train_raw)
X_test = preprocessor.transform(X_test_raw)

```



```

# 7. Salvar objetos que serão usados nas próximas etapas
X_train.shape
X_test.shape
y_train.shape

# 8. Armazenar IDs do teste para submissão futura
test_ids = df_test['id'].copy()

# 9. Exibir informações resumidas
print('Shape X_train:', X_train.shape)
print('Shape X_test:', X_test.shape)
print('Shape y_train:', y_train.shape)
print('Número de colunas de alta cardinalidade:', len(high_cardinality_cols))
print('Número de colunas de baixa cardinalidade:', len(low_cardinality_cols))

```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[4], line 39
    31 low_card_pipeline = Pipeline([
    32     ('imputer', SimpleImputer(strategy='most_frequent')),
    33     ('ordinal', OrdinalEncoder(handle_unknown='use_encoded_value',
↳ unknown_value=-1))
    34 ])
    36 # 4.3 Pipeline para colunas categóricas de alta cardinalidade
    37 high_card_pipeline = Pipeline([
    38     ('imputer', SimpleImputer(strategy='most_frequent')),
----> 39     ('onehot', OneHotEncoder(handle_unknown='ignore', sparse=False))
    40 ])
    42 # 5. Combinar tudo em um ColumnTransformer
    43 preprocessor = ColumnTransformer(
    44     transformers=[
    45         ('num', numeric_pipeline, numeric_cols),
    (...
    48     ]
    49 )

TypeError: OneHotEncoder.__init__() got an unexpected keyword argument 'sparse'

```

```

[5]: # Corrigindo o pipeline de pré-processamento (T3)
# -----
# Importações necessárias
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder, StandardScaler

```

```

# 1. Separar target e features
y_train = df_train['Depressão'].copy()
X_train_raw = df_train.drop(columns=['Depressão'])
X_test_raw = df_test.copy()

# 2. Identificar colunas numéricas e categóricas
numeric_cols = X_train_raw.select_dtypes(include=['number']).columns.tolist()
categorical_cols = X_train_raw.select_dtypes(include=['object', 'category',
↳ 'bool']).columns.tolist()

# 3. Definir limiar de cardinalidade
cardinality_threshold = 10
low_cardinality_cols = [col for col in categorical_cols if X_train_raw[col].
↳ nunique(dropna=False) <= cardinality_threshold]
high_cardinality_cols = [col for col in categorical_cols if X_train_raw[col].
↳ nunique(dropna=False) > cardinality_threshold]

# 4. Criar pipelines para cada tipo de coluna
# 4.1 Pipeline para colunas numéricas
numeric_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# 4.2 Pipeline para colunas categóricas de baixa cardinalidade
low_card_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('ordinal', OrdinalEncoder(handle_unknown='use_encoded_value',
↳ unknown_value=-1))
])

# 4.3 Pipeline para colunas categóricas de alta cardinalidade
# Usamos 'sparse_output' para garantir compatibilidade com scikit-learn 1.3+.
high_card_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

# 5. Combinar tudo em um ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_pipeline, numeric_cols),
        ('low_cat', low_card_pipeline, low_cardinality_cols),
        ('high_cat', high_card_pipeline, high_cardinality_cols)
    ]
)

```

```

# 6. Fit no conjunto de treino e transformar ambos os conjuntos
X_train = preprocessor.fit_transform(X_train_raw)
X_test = preprocessor.transform(X_test_raw)

# 7. Armazenar IDs do teste para submissão futura
test_ids = df_test['id'].copy()

# 8. Exibir informações resumidas
print('Shape X_train:', X_train.shape)
print('Shape X_test:', X_test.shape)
print('Shape y_train:', y_train.shape)
print('Número de colunas de alta cardinalidade:', len(high_cardinality_cols))
print('Número de colunas de baixa cardinalidade:', len(low_cardinality_cols))

```

```

Shape X_train: (140700, 769)
Shape X_test: (93800, 769)
Shape y_train: (140700,)
Número de colunas de alta cardinalidade: 6
Número de colunas de baixa cardinalidade: 4

```

1.4 Etapa 4 – Engenharia de Features

1.4.1 Visão geral do que já foi feito

- **Carregamento:** os arquivos `train.csv` e `test.csv` foram lidos em `df_train` e `df_test`.
- **EDA:** verificamos tipos de dados, distribuição da variável alvo, estatísticas descritivas, correlação e porcentagem de valores ausentes.
- **Pré-processamento:** aplicamos imputação, codificação (Ordinal e OneHot) e escalonamento numérico, resultando em `X_train` ($140\,700 \times 769$) e `X_test` ($93\,800 \times 769$).

1.4.2 Objetivo desta etapa (T4)

Criar *features adicionais* que possam capturar relações não lineares, interações entre variáveis e padrões de comportamento que o modelo base (XGBoost) possa explorar. Essas features devem ser geradas **apenas a partir das colunas originais** – não podemos usar a coluna alvo **Depressão** para criar novas variáveis.

1.4.3 Estratégias de feature engineering

1. **Interações** – multiplicar ou somar variáveis que têm sentido biológico ou social, por exemplo:
 - Pressão Acadêmica * Satisfação com os Estudos
 - Pressão no Trabalho * Satisfação com o Trabalho
 - Horas de Trabalho/Estudo * Idade (para capturar a carga de trabalho em diferentes faixas etárias).
2. **Binarizações** – transformar variáveis contínuas em categorias binárias ou discretas:
 - Idade em faixas (18-25, 26-35, 36-45, 46-55, 56+).
 - Horas de Trabalho/Estudo em “Alta” (>8h) vs “Baixa” (<8h).

3. **Agregações** – criar estatísticas agregadas a partir de colunas de alta cardinalidade, por exemplo:
 - Contagem de palavras distintas em Nome (para medir complexidade do nome).
 - Média de Pressão Acadêmica por Cidade (para capturar pressão regional).
4. **Transformações matemáticas** – aplicar log, raiz quadrada ou normalização de variáveis que apresentam distribuição assimétrica.
5. **Combinação de categorias** – agrupar valores raros em uma categoria “Outros” para reduzir a cardinalidade sem perder informação.

1.4.4 Como será implementado

- **Cópia:** antes de criar as novas colunas, faremos `df_train_fe = df_train.copy()` e `df_test_fe = df_test.copy()` para preservar os dados originais.
- **Aplicação consistente:** todas as transformações serão aplicadas simultaneamente a `df_train_fe` e `df_test_fe` para evitar vazamento de informação.
- **Integração com o pipeline:** após a criação das novas colunas, o conjunto resultante será concatenado com o output do **preprocessor** (`X_train`, `X_test`) usando `hstack` ou `np.concatenate`.
- **Persistência:** as matrizes finais serão armazenadas em variáveis `X_train_fe` e `X_test_fe` que serão usadas nas próximas etapas (T5-T10).

1.4.5 Próximos passos

1. Implementar as transformações descritas acima em uma célula de código (T4).
2. Verificar a dimensionalidade resultante e garantir que não haja colunas vazias.
3. Continuar com a divisão em treino/validação (T5) e treinamento do modelo base (T6).

Com a engenharia de features concluída, estaremos prontos para melhorar a performance do modelo e gerar a submissão inicial.

```
[6]: # Etapa 4 - Engenharia de Features (T4)
# -----
# 1. Fazer cópias dos DataFrames originais para preservar os dados
df_train_fe = df_train.copy()
df_test_fe = df_test.copy()

# 2. Interações entre variáveis numéricas
df_train_fe['PressãoAcad_SatEstudos'] = df_train_fe['Pressão Acadêmica'] *
    ↪df_train_fe['Satisfação com os Estudos']
df_test_fe['PressãoAcad_SatEstudos'] = df_test_fe['Pressão Acadêmica'] *
    ↪df_test_fe['Satisfação com os Estudos']

df_train_fe['PressãoTrb_SatTrb'] = df_train_fe['Pressão no Trabalho'] *
    ↪df_train_fe['Satisfação com o Trabalho']
df_test_fe['PressãoTrb_SatTrb'] = df_test_fe['Pressão no Trabalho'] *
    ↪df_test_fe['Satisfação com o Trabalho']
```

```

df_train_fe['HorasIdade'] = df_train_fe['Horas de Trabalho/Estudo'] *
    ↪df_train_fe['Idade']
df_test_fe['HorasIdade'] = df_test_fe['Horas de Trabalho/Estudo'] *
    ↪df_test_fe['Idade']

# 3. Binarizações e discretizações
# 3.1 Faixas de idade (5 categorias)
age_bins = [0, 25, 35, 45, 55, 200]
age_labels = [0, 1, 2, 3, 4]
df_train_fe['Idade_bin'] = pd.cut(df_train_fe['Idade'], bins=age_bins,
    ↪labels=age_labels, right=False).astype(int)
df_test_fe['Idade_bin'] = pd.cut(df_test_fe['Idade'], bins=age_bins,
    ↪labels=age_labels, right=False).astype(int)
# 3.2 Alta carga de trabalho (>8h)
df_train_fe['HighWorkload'] = (df_train_fe['Horas de Trabalho/Estudo'] > 8).
    ↪astype(int)
df_test_fe['HighWorkload'] = (df_test_fe['Horas de Trabalho/Estudo'] > 8).
    ↪astype(int)

# 4. Agregações
# 4.1 Contagem de palavras no nome
df_train_fe['Nome_word_count'] = df_train_fe['Nome'].str.split().apply(len)
df_test_fe['Nome_word_count'] = df_test_fe['Nome'].str.split().apply(len)
# 4.2 Média de Pressão Acadêmica por Cidade (usando treino para evitar leakage)
city_pressure_mean = df_train_fe.groupby('Cidade')['Pressão Acadêmica'].
    ↪transform('mean')
df_train_fe['CityPressMean'] = city_pressure_mean
city_pressure_mean_test = df_test_fe['Cidade'].map(df_train_fe.
    ↪groupby('Cidade')['Pressão Acadêmica'].mean())
df_test_fe['CityPressMean'] = city_pressure_mean_test
# Preencher valores ausentes (caso cidade não apareça no treino) com média
    ↪global
global_press_mean = df_train_fe['Pressão Acadêmica'].mean()
df_test_fe['CityPressMean'].fillna(global_press_mean, inplace=True)

# 5. Transformações matemáticas
df_train_fe['log_PressãoAcad'] = np.log1p(df_train_fe['Pressão Acadêmica'])
df_test_fe['log_PressãoAcad'] = np.log1p(df_test_fe['Pressão Acadêmica'])

df_train_fe['sqrt_Horas'] = np.sqrt(df_train_fe['Horas de Trabalho/Estudo'])
df_test_fe['sqrt_Horas'] = np.sqrt(df_test_fe['Horas de Trabalho/Estudo'])

# 6. Selecionar apenas as novas colunas criadas
new_feature_cols = [
    'PressãoAcad_SatEstudos',
    'PressãoTrb_SatTrb',

```

```

    'HorasIdade',
    'Idade_bin',
    'HighWorkload',
    'Nome_word_count',
    'CityPressMean',
    'log_PressãoAcad',
    'sqrt_Horas'
]

# 7. Converter as novas features para arrays numpy
train_new_feats = df_train_fe[new_feature_cols].values
test_new_feats = df_test_fe[new_feature_cols].values

# 8. Concatenar com as matrizes já processadas (X_train, X_test)
from scipy import sparse
X_train_fe = sparse.hstack([X_train, train_new_feats])
X_test_fe = sparse.hstack([X_test, test_new_feats])

# 9. Informações resumidas
print('Shape X_train_fe:', X_train_fe.shape)
print('Shape X_test_fe:', X_test_fe.shape)
print('Número de novas features adicionadas:', len(new_feature_cols))

```

/tmp/ipykernel_90842/2544795732.py:38: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df_test_fe['CityPressMean'].fillna(global_press_mean, inplace=True)
```

```
-----
ValueError                                Traceback (most recent call last)
```

```
Cell In[6], line 66
```

```

    64 # 8. Concatenar com as matrizes já processadas (X_train, X_test)
    65 from scipy import sparse
--> 66 X_train_fe = sparse.hstack([X_train, train_new_feats])
    67 X_test_fe = sparse.hstack([X_test, test_new_feats])
    69 # 9. Informações resumidas

```

```

File ~/Mestrado/Projeto/Mestrado-Privado/p3_11-venv/lib/python3.11/site-package /
↳scipy/sparse/_construct.py:797, in hstack(blocks, format, dtype)

```

```

757 def hstack(blocks, format=None, dtype=None):
758     """
759     Stack sparse matrices horizontally (column wise)
760     (...)
795
796     """
--> 797     blocks = np.asarray(blocks, dtype='object')
798     if any(isinstance(b, sparray) for b in blocks.flat):
799         return _block([blocks], format, dtype)

```

ValueError: could not broadcast input array from shape (140700,769) into shape (140700,)

```

[7]: # Etapa 4 - Engenharia de Features (T4) - correção da concatenação
# -----
# 1. Fazer cópias dos DataFrames originais para preservar os dados
df_train_fe = df_train.copy()
df_test_fe = df_test.copy()

# 2. Interações entre variáveis numéricas
df_train_fe['PressãoAcad_SatEstudos'] = df_train_fe['Pressão Acadêmica'] *
    ↳df_train_fe['Satisfação com os Estudos']
df_test_fe['PressãoAcad_SatEstudos'] = df_test_fe['Pressão Acadêmica'] *
    ↳df_test_fe['Satisfação com os Estudos']

df_train_fe['PressãoTrb_SatTrb'] = df_train_fe['Pressão no Trabalho'] *
    ↳df_train_fe['Satisfação com o Trabalho']
df_test_fe['PressãoTrb_SatTrb'] = df_test_fe['Pressão no Trabalho'] *
    ↳df_test_fe['Satisfação com o Trabalho']

df_train_fe['HorasIdade'] = df_train_fe['Horas de Trabalho/Estudo'] *
    ↳df_train_fe['Idade']
df_test_fe['HorasIdade'] = df_test_fe['Horas de Trabalho/Estudo'] *
    ↳df_test_fe['Idade']

# 3. Binarizações e discretizações
# 3.1 Faixas de idade (5 categorias)
age_bins = [0, 25, 35, 45, 55, 200]
age_labels = [0, 1, 2, 3, 4]
df_train_fe['Idade_bin'] = pd.cut(df_train_fe['Idade'], bins=age_bins,
    ↳labels=age_labels, right=False).astype(int)
df_test_fe['Idade_bin'] = pd.cut(df_test_fe['Idade'], bins=age_bins,
    ↳labels=age_labels, right=False).astype(int)
# 3.2 Alta carga de trabalho (>8h)

```

```

df_train_fe['HighWorkload'] = (df_train_fe['Horas de Trabalho/Estudo'] > 8).
    ↳astype(int)
df_test_fe['HighWorkload'] = (df_test_fe['Horas de Trabalho/Estudo'] > 8).
    ↳astype(int)

# 4. Agregações
# 4.1 Contagem de palavras no nome
df_train_fe['Nome_word_count'] = df_train_fe['Nome'].str.split().apply(len)
df_test_fe['Nome_word_count'] = df_test_fe['Nome'].str.split().apply(len)
# 4.2 Média de Pressão Acadêmica por Cidade (usando treino para evitar leakage)
city_pressure_mean = df_train_fe.groupby('Cidade')['Pressão Acadêmica'].
    ↳transform('mean')
df_train_fe['CityPressMean'] = city_pressure_mean
city_pressure_mean_test = df_test_fe['Cidade'].map(df_train_fe.
    ↳groupby('Cidade')['Pressão Acadêmica'].mean())
df_test_fe['CityPressMean'] = city_pressure_mean_test
# Preencher valores ausentes (caso cidade não apareça no treino) com média
    ↳global
global_press_mean = df_train_fe['Pressão Acadêmica'].mean()
df_test_fe['CityPressMean'].fillna(global_press_mean, inplace=True)

# 5. Transformações matemáticas
df_train_fe['log_PressãoAcad'] = np.log1p(df_train_fe['Pressão Acadêmica'])
df_test_fe['log_PressãoAcad'] = np.log1p(df_test_fe['Pressão Acadêmica'])

df_train_fe['sqrt_Horas'] = np.sqrt(df_train_fe['Horas de Trabalho/Estudo'])
df_test_fe['sqrt_Horas'] = np.sqrt(df_test_fe['Horas de Trabalho/Estudo'])

# 6. Selecionar apenas as novas colunas criadas
new_feature_cols = [
    'PressãoAcad_SatEstudos',
    'PressãoTrb_SatTrb',
    'HorasIdade',
    'Idade_bin',
    'HighWorkload',
    'Nome_word_count',
    'CityPressMean',
    'log_PressãoAcad',
    'sqrt_Horas'
]

# 7. Converter as novas features para arrays numpy e garantir 2D
train_new_feats = df_train_fe[new_feature_cols].values
test_new_feats = df_test_fe[new_feature_cols].values
if train_new_feats.ndim == 1:
    train_new_feats = train_new_feats.reshape(-1, 1)
if test_new_feats.ndim == 1:

```



```

test_new_feats = test_new_feats.reshape(-1, 1)

# 8. Concatenar com as matrizes já processadas (X_train, X_test) usando numpy
↳ hstack
X_train_fe = np.hstack([X_train, train_new_feats])
X_test_fe = np.hstack([X_test, test_new_feats])

# 9. Informações resumidas
print('Shape X_train_fe:', X_train_fe.shape)
print('Shape X_test_fe:', X_test_fe.shape)
print('Número de novas features adicionadas:', len(new_feature_cols))

```

Shape X_train_fe: (140700, 778)

Shape X_test_fe: (93800, 778)

Número de novas features adicionadas: 9

/tmp/ipykernel_90842/1799325371.py:38: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df_test_fe['CityPressMean'].fillna(global_press_mean, inplace=True)
```

1.5 Etapa 5 – Separação em Treino e Validação (T5)

1.5.1 Visão geral do que já foi feito

- **Carregamento:** train.csv e test.csv foram lidos em df_train e df_test.
- **EDA:** tipos de dados, distribuição da variável alvo, estatísticas descritivas, correlação e valores ausentes foram analisados.
- **Pré-processamento:** imputação, codificação (Ordinal e OneHot) e escalonamento numérico foram aplicados, gerando X_train (140 700 × 769) e X_test (93 800 × 769).
- **Engenharia de Features:** nove features adicionais foram criadas e concatenadas, resultando em X_train_fe (140 700 × 778) e X_test_fe (93 800 × 778).

1.5.2 Objetivo desta etapa

Dividir o conjunto de treinamento em duas partes: **treino** (80%) e **validação** (20%) usando train_test_split do scikit-learn. Essa divisão será usada para avaliar o modelo base antes de treinar no conjunto completo.

1.5.3 Estratégia de divisão

- **Random state:** 42 (para reprodutibilidade).
- **Stratify:** a coluna alvo **Depressão** será usada para manter a mesma proporção de classes em ambos os subconjuntos, evitando viés de desbalanceamento.
- **Shuffle:** habilitado (padrão) para garantir que a amostra seja aleatória.
- **Output:** X_train_split, X_val_split, y_train_split, y_val_split.

1.5.4 Próximos passos

1. **Treinar** o modelo base (XGBoost) usando X_train_split e y_train_split.
2. **Avaliar** a acurácia no conjunto de validação (X_val_split, y_val_split).
3. Se a performance for satisfatória, prosseguir para a etapa de otimização de hiper-parâmetros (T8) e, finalmente, gerar a submissão inicial (T7).

Esta célula marca a transição do pré-processamento para a fase de modelagem, garantindo que a avaliação seja feita de forma justa e sem vazamento de informação.

```
[8]: # Etapa 5 - Separação em Treino e Validação (T5)
# -----
from sklearn.model_selection import train_test_split

# Dividir o conjunto de treinamento em 80/20 com stratificação
X_train_split, X_val_split, y_train_split, y_val_split = train_test_split(
    X_train_fe,
    y_train,
    test_size=0.20,
    random_state=42,
    stratify=y_train
)

# Exibir dimensões dos subconjuntos
print('Shape X_train_split:', X_train_split.shape)
print('Shape X_val_split:', X_val_split.shape)
print('Shape y_train_split:', y_train_split.shape)
print('Shape y_val_split:', y_val_split.shape)
```

```
Shape X_train_split: (112560, 778)
```

```
Shape X_val_split: (28140, 778)
```

```
Shape y_train_split: (112560,)
```

```
Shape y_val_split: (28140,)
```

1.6 Etapa 6 – Treinamento do Modelo Base (XGBoost)

1.6.1 Visão geral do que já foi feito

- **Pré-processamento:** os dados foram limpos, codificados e normalizados, resultando em X_train_fe ($140\,700 \times 778$) e X_test_fe ($93\,800 \times 778$).
- **Divisão:** o conjunto de treino foi separado em treino (80%) e validação (20%) com train_test_split, mantendo a proporção de classes (stratify=y_train).

1.6.2 Objetivo desta célula

Treinar um modelo base usando **XGBoostClassifier** com os parâmetros padrão, avaliar a acurácia no conjunto de validação e gerar as predições iniciais para o conjunto de teste, salvando-as em `submission.csv`.

1.6.3 Processo a ser executado

1. **Instanciar** o classificador: `XGBClassifier(n_estimators=100, max_depth=6, learning_rate=0.1, subsample=1.0, colsample_bytree=1.0, objective='binary:logistic', eval_metric='logloss', n_jobs=16, random_state=42)` – esses são os valores padrão do XGBoost, mas explicitamos para garantir consistência.
2. **Treinar** o modelo com `X_train_split` e `y_train_split`.
3. **Prever** as classes no conjunto de validação (`X_val_split`) usando `predict`. Como o objetivo é acurácia, converteremos as probabilidades em rótulos binários (0/1) com limiar 0.5.
4. **Avaliar** a acurácia com `accuracy_score(y_val_split, y_val_pred)` e imprimir o resultado.
5. **Prever** as classes para o conjunto de teste (`X_test_fe`).
6. **Criar** o DataFrame de submissão: `pd.DataFrame({'id': test_ids, 'Depressão': y_test_pred})`.
7. **Salvar** em `submission.csv` usando `to_csv('submission.csv', index=False)`.

1.6.4 Observações

- O modelo será treinado apenas no conjunto de treino (80 %) para evitar vazamento de informação.
- Usamos 16 threads (`n_jobs=16`) para acelerar o treinamento dentro do limite de tempo.
- A acurácia obtida no conjunto de validação servirá como referência para a etapa de otimização de hiper-parâmetros (T8).
- Caso a acurácia seja inferior ao esperado, podemos ajustar parâmetros como `max_depth`, `learning_rate` ou `n_estimators` na próxima etapa.
- Se houver algum erro de execução (por exemplo, falta de memória ou erro de importação), verifique se o pacote `xgboost` está instalado e se a versão é compatível com o `scikit-learn`.

Com isso, teremos o arquivo `submission.csv` inicial pronto para ser enviado ao Kaggle, e a base para compararmos melhorias nas etapas seguintes.

```
[9]: # Etapa 6 - Treinamento do Modelo Base (XGBoost)
# -----
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

# Instanciar o classificador com parâmetros padrão (explicitados para clareza)
xgb_clf = XGBClassifier(
    n_estimators=100,
    max_depth=6,
    learning_rate=0.1,
    subsample=1.0,
```

```

        colsample_bytree=1.0,
        objective='binary:logistic',
        eval_metric='logloss',
        n_jobs=16,
        random_state=42
    )

    # Treinar no conjunto de treino (80%)
    xgb_clf.fit(X_train_split, y_train_split)

    # Previsões no conjunto de validação
    y_val_pred = xgb_clf.predict(X_val_split)

    # Avaliar acurácia
    val_accuracy = accuracy_score(y_val_split, y_val_pred)
    print(f'Acurácia no conjunto de validação: {val_accuracy:.4f}')

    # Previsões para o conjunto de teste
    y_test_pred = xgb_clf.predict(X_test_fe)

    # Criar DataFrame de submissão
    submission = pd.DataFrame({
        'id': test_ids,
        'Depressão': y_test_pred.astype(int)
    })

    # Salvar em submission.csv
    submission.to_csv('submission.csv', index=False)
    print('Arquivo submission.csv criado com sucesso!')

```

Acurácia no conjunto de validação: 0.9388

Arquivo submission.csv criado com sucesso!

1.7 Etapa 7 – Avaliação do Modelo Base e Submissão Inicial

1.7.1 Visão geral do que já foi feito

- **Pré-processamento:** os dados foram limpos, codificados e normalizados, resultando em `X_train_fe` ($140\,700 \times 778$) e `X_test_fe` ($93\,800 \times 778$).
- **Divisão:** o conjunto de treino foi separado em treino (80%) e validação (20%) com `train_test_split`, mantendo a proporção de classes (`stratify=y_train`).
- **Modelo Base:** um `XGBClassifier` com parâmetros padrão (100 estimadores, profundidade 6, taxa de aprendizado 0.1) foi treinado apenas no conjunto de treino.
- **Avaliação:** a acurácia no conjunto de validação foi de **0.9388**, muito acima da acurácia base (0.818) e satisfatória para a competição.
- **Submissão:** as previsões para o conjunto de teste foram geradas e salvas em `submission.csv` com as colunas `id`, `Depressão`.

1.7.2 Próximos passos

1. **Otimização de hiper-parâmetros** (T8) – usar `RandomizedSearchCV` ou `GridSearchCV` para melhorar ainda mais a acurácia.
2. **Treinar modelo final** no conjunto completo com os melhores parâmetros (T9).
3. **Gerar submissão final** (T10) e enviar ao Kaggle.

Esta célula resume a etapa de avaliação e garante que o arquivo `submission.csv` inicial esteja pronto para ser usado como baseline. Se precisar ajustar algo no pipeline ou nos hiper-parâmetros, basta voltar às etapas anteriores e re-executar.

```
[10]: from sklearn.metrics import accuracy_score
import pandas as pd

# Avaliar o modelo base no conjunto de validação
val_pred = xgb_clf.predict(X_val_split)
val_acc = accuracy_score(y_val_split, val_pred)
print(f'Acurácia no conjunto de validação: {val_acc:.4f}')

# Previsões para o conjunto de teste
test_pred = xgb_clf.predict(X_test_fe)

# Criar DataFrame de submissão
submission = pd.DataFrame({
    'id': test_ids,
    'Depressão': test_pred.astype(int)
})

# Salvar em submission.csv
submission.to_csv('submission.csv', index=False)
print('Arquivo submission.csv criado com sucesso!')
```

Acurácia no conjunto de validação: 0.9388

Arquivo submission.csv criado com sucesso!

1.8 Etapa 8 – Otimização de Hiperparâmetros (T8)

1.8.1 Visão geral do que já foi feito

- **Pré-processamento:** os dados foram limpos, codificados e normalizados, resultando em `X_train_fe` ($140\,700 \times 778$) e `X_test_fe` ($93\,800 \times 778$).
- **Divisão:** o conjunto de treino foi separado em treino (80%) e validação (20%) com `train_test_split`, mantendo a proporção de classes (`stratify=y_train`).
- **Modelo Base:** um `XGBClassifier` com parâmetros padrão (100 estimadores, profundidade 6, taxa de aprendizado 0.1) foi treinado apenas no conjunto de treino.
- **Avaliação:** a acurácia no conjunto de validação foi de **0.9388**, muito acima da acurácia base (0.818) e satisfatória para a competição.
- **Submissão Inicial:** as previsões para o conjunto de teste foram geradas e salvas em `submission.csv`.

1.8.2 Objetivo desta etapa

Melhorar ainda mais a acurácia do modelo base ajustando os hiperparâmetros do `XGBClassifier`. Para isso utilizaremos `RandomizedSearchCV` (ou `GridSearchCV` se o tempo permitir) com validação cruzada estratificada de 5 folds, mantendo o mesmo `random_state=42` e `n_jobs=16` para aproveitar os 16 núcleos disponíveis.

1.8.3 Estratégia de busca

1. **Definir o espaço de parâmetros:** incluiremos variáveis que costumam ter grande impacto em modelos de árvore, como `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, `gamma`, `min_child_weight` e `reg_alpha/reg_lambda`.
2. **Limitar o número de combinações:** para respeitar o limite de 60 minutos, usaremos `n_iter=50` no `RandomizedSearchCV`. Isso garante cobertura suficiente do espaço sem sobrecarregar o tempo de execução.
3. **Validação cruzada estratificada:** `cv=5` com `StratifiedKFold` assegura que cada fold mantenha a mesma proporção de classes, evitando viés de desbalanceamento.
4. **Métrica de avaliação:** usaremos `accuracy` como métrica de scoring, pois é a métrica oficial da competição.
5. **Early stopping:** opcionalmente, podemos habilitar `early_stopping_rounds` dentro do `XGBoost` para evitar overfitting, mas isso requer um conjunto de validação separado; para simplicidade, manteremos apenas a busca de hiperparâmetros.

1.8.4 Passos a serem executados (em código, que será inserido em uma célula de código subsequente)

- Instanciar o `XGBClassifier` com `n_jobs=16` e `random_state=42`.
- Definir o dicionário `param_distributions` com os parâmetros listados acima.
- Criar o objeto `RandomizedSearchCV` com `n_iter=50`, `cv=5`, `scoring='accuracy'`, `verbose=1` e `n_jobs=16`.
- Ajustar o objeto ao conjunto de treino (`X_train_split`, `y_train_split`).
- Exibir os melhores parâmetros e a melhor acurácia obtida na validação cruzada.
- Salvar o modelo final (com os melhores parâmetros) em uma variável `best_xgb` para uso nas próximas etapas (T9 e T10).

1.8.5 Observações importantes

- **Tempo:** com 50 iterações e 5 folds, a busca deve ser concluída em poucos minutos, bem dentro do limite de 60 minutos.
- **Reprodutibilidade:** manter `random_state=42` em todas as etapas garante que os resultados sejam consistentes.
- **Persistência:** após a busca, guardaremos os melhores parâmetros em um dicionário `best_params` e o modelo em `best_xgb` para posterior treinamento no conjunto completo.
- **Validação:** a acurácia final do modelo otimizado será comparada com a base (0.9388). Se houver ganho, prosseguiremos para a etapa T9.

Esta célula descreve a lógica e os passos que serão executados na próxima célula de código. Ao final, teremos um modelo `XGBoost` com hiperparâmetros otimizados, pronto para ser treinado no conjunto completo e gerar a submissão final.

```
[11]: # Etapa 8 - Otimização de Hiperparâmetros (RandomizedSearchCV)
# -----
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from xgboost import XGBClassifier
import numpy as np

# Definir o classificador base com n_jobs e random_state fixos
xgb_base = XGBClassifier(
    n_jobs=16,
    random_state=42,
    objective='binary:logistic',
    eval_metric='logloss'
)

# Espaço de parâmetros para a busca aleatória
param_distributions = {
    'n_estimators': [100, 200, 300, 400, 500],
    'max_depth': [3, 4, 5, 6, 7, 8],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'gamma': [0, 0.1, 0.2, 0.5],
    'min_child_weight': [1, 3, 5],
    'reg_alpha': [0, 0.01, 0.1],
    'reg_lambda': [1, 1.5, 2]
}

# Configurar o RandomizedSearchCV
random_search = RandomizedSearchCV(
    estimator=xgb_base,
    param_distributions=param_distributions,
    n_iter=50,
    scoring='accuracy',
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    verbose=1,
    n_jobs=16,
    random_state=42
)

# Executar a busca no conjunto de treino (80%)
random_search.fit(X_train_split, y_train_split)

# Exibir os melhores parâmetros e a melhor acurácia
print('Melhores parâmetros encontrados:')
print(random_search.best_params_)
print(f"Melhor acurácia (CV): {random_search.best_score_:.4f}")
```

```

# Salvar o melhor estimador para uso posterior
best_xgb = random_search.best_estimator_
best_params = random_search.best_params_

# Opcional: re-treinar o melhor modelo no conjunto completo (80%) para garantir
#   ↳ que ele use o mesmo número de estimadores
# (não é obrigatório, mas pode ser útil antes da etapa final)
# best_xgb.fit(X_train_split, y_train_split)

# Guardar o modelo em disco (opcional)
# import joblib
# joblib.dump(best_xgb, 'best_xgb_model.joblib')

```

Fitting 5 folds for each of 50 candidates, totalling 250 fits

```

[ ]: # Etapa 8 - Otimização de Hiperparâmetros (RandomizedSearchCV) - versão
#   ↳ otimizada
# -----
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from xgboost import XGBClassifier
import numpy as np

# Base classifier with fixed random state and limited threads
xgb_base = XGBClassifier(
    n_jobs=8,                # reduzir o número de threads para evitar
    ↳ saturação de memória
    random_state=42,
    objective='binary:logistic',
    eval_metric='logloss'
)

# Espaço de parâmetros reduzido para manter o tempo de busca dentro do limite
param_distributions = {
    'n_estimators': [100, 200, 300],
    'max_depth': [3, 4, 5, 6, 7],
    'learning_rate': [0.05, 0.1, 0.2],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0],
    'gamma': [0, 0.1],
    'min_child_weight': [1, 3],
    'reg_alpha': [0, 0.01, 0.1],
    'reg_lambda': [1, 1.5, 2]
}

# Configurar RandomizedSearchCV com 3 folds e 20 iterações
random_search = RandomizedSearchCV(
    estimator=xgb_base,

```



```

param_distributions=param_distributions,
n_iter=20,
scoring='accuracy',
cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=42),
verbose=0,
n_jobs=1,          # cada estimador já usa 8 threads, então não
↳paralelizamos o grid
    random_state=42
)

# Executar a busca no conjunto de treino (80%)
random_search.fit(X_train_split, y_train_split)

# Exibir os melhores parâmetros e a melhor acurácia
print('Melhores parâmetros encontrados:')
print(random_search.best_params_)
print(f"Melhor acurácia (CV): {random_search.best_score_:.4f}")

# Salvar o melhor estimador para uso posterior
best_xgb = random_search.best_estimator_
best_params = random_search.best_params_

# Re-treinar o melhor modelo no conjunto completo de treino (80%) para garantir
↳que ele use o mesmo número de estimadores
best_xgb.fit(X_train_split, y_train_split)

# Avaliar no conjunto de validação
val_pred = best_xgb.predict(X_val_split)
val_acc = accuracy_score(y_val_split, val_pred)
print(f'Acurácia do melhor modelo no conjunto de validação: {val_acc:.4f}')

# Previsões para o conjunto de teste
test_pred = best_xgb.predict(X_test_fe)

# Criar DataFrame de submissão
submission = pd.DataFrame({
    'id': test_ids,
    'Depressão': test_pred.astype(int)
})

# Salvar em submission.csv
submission.to_csv('submission.csv', index=False)
print('Arquivo submission.csv criado com sucesso!')

```

```

[1]: # Etapa 8 - Otimização de Hiperparâmetros (RandomizedSearchCV) - versão
↳otimizada e segura
# -----

```

```

from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score
import numpy as np

# Base classifier with a fast tree method and limited threads to keep memory
↳usage low
xgb_base = XGBClassifier(
    n_jobs=4,                # 4 threads para evitar saturação de memória
    random_state=42,
    objective='binary:logistic',
    eval_metric='logloss',
    use_label_encoder=False,
    verbosity=0,
    tree_method='hist'      # método histórico é mais rápido e consome menos
↳memória
)

# Espaço de parâmetros reduzido para manter o tempo de busca dentro do limite
param_distributions = {
    'n_estimators': [100, 150, 200],
    'max_depth': [3, 4, 5, 6],
    'learning_rate': [0.05, 0.1, 0.2],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0],
    'gamma': [0, 0.1],
    'min_child_weight': [1, 3],
    'reg_alpha': [0, 0.01],
    'reg_lambda': [1, 1.5]
}

# Configurar RandomizedSearchCV com 3 folds e 10 iterações (total 30 modelos)
random_search = RandomizedSearchCV(
    estimator=xgb_base,
    param_distributions=param_distributions,
    n_iter=10,
    scoring='accuracy',
    cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=42),
    verbose=0,
    n_jobs=1,                # cada estimador já usa 4 threads, então não
↳paralelizamos o grid
    random_state=42
)

# Executar a busca no conjunto de treino (80%)
random_search.fit(X_train_split, y_train_split)

```

```

# Exibir os melhores parâmetros e a melhor acurácia
print('Melhores parâmetros encontrados:')
print(random_search.best_params_)
print(f"Melhor acurácia (CV): {random_search.best_score_:.4f}")

# Salvar o melhor estimador para uso posterior
best_xgb = random_search.best_estimator_
best_params = random_search.best_params_

# Avaliar o melhor modelo no conjunto de validação
val_pred = best_xgb.predict(X_val_split)
val_acc = accuracy_score(y_val_split, val_pred)
print(f'Acurácia do melhor modelo no conjunto de validação: {val_acc:.4f}')

# Previsões para o conjunto de teste
test_pred = best_xgb.predict(X_test_fe)

# Criar DataFrame de submissão
submission = pd.DataFrame({
    'id': test_ids,
    'Depressão': test_pred.astype(int)
})

# Salvar em submission.csv
submission.to_csv('submission.csv', index=False)
print('Arquivo submission.csv criado com sucesso!')

```

/home/jao/Mestrado/Projeto/Mestrado-Privado/p3_11-venv/lib/python3.11/site-packages/xgboost/sklearn.py:1421: UserWarning: `use\_label\_encoder` is deprecated in 1.7.0.

warnings.warn("`use\_label\_encoder` is deprecated in 1.7.0.")

```

-----
NameError                                Traceback (most recent call last)
Cell In[1], line 45
    33 random_search = RandomizedSearchCV(
    34     estimator=xgb_base,
    35     param_distributions=param_distributions,
    (...)
    41     random_state=42
    42 )
    44 # Executar a busca no conjunto de treino (80%)
--> 45 random_search.fit(X_train_split, y_train_split)
    47 # Exibir os melhores parâmetros e a melhor acurácia
    48 print('Melhores parâmetros encontrados:')

NameError: name 'X_train_split' is not defined

```

```

[2]: # Re-build the pipeline and run a quick hyper-parameter search
import pandas as pd
import numpy as np
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split, RandomizedSearchCV,
↳StratifiedKFold
from sklearn.metrics import accuracy_score
from xgboost import XGBClassifier

# 1. Load data
df_train = pd.read_csv('train.csv')
df_test = pd.read_csv('test.csv')

# 2. Separate target and features
y_train = df_train['Depressão'].copy()
X_train_raw = df_train.drop(columns=['Depressão'])
X_test_raw = df_test.copy()

# 3. Identify numeric and categorical columns
numeric_cols = X_train_raw.select_dtypes(include=['number']).columns.tolist()
categorical_cols = X_train_raw.select_dtypes(include=['object', 'category',
↳'bool']).columns.tolist()

# 4. Cardinality split
cardinality_threshold = 10
low_cardinality_cols = [c for c in categorical_cols if X_train_raw[c].
↳nunique(dropna=False) <= cardinality_threshold]
high_cardinality_cols = [c for c in categorical_cols if X_train_raw[c].
↳nunique(dropna=False) > cardinality_threshold]

# 5. Pipelines for each type
numeric_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
low_card_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('ordinal', OrdinalEncoder(handle_unknown='use_encoded_value',
↳unknown_value=-1))
])
high_card_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

```

```

# 6. Combine with ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_pipeline, numeric_cols),
        ('low_cat', low_card_pipeline, low_cardinality_cols),
        ('high_cat', high_card_pipeline, high_cardinality_cols)
    ]
)

# 7. Feature engineering (same as T4)
df_train_fe = df_train.copy()
df_test_fe = df_test.copy()

# Interactions
df_train_fe['PressãoAcad_SatEstudos'] = df_train_fe['Pressão Acadêmica'] *
    ↪df_train_fe['Satisfação com os Estudos']
df_test_fe['PressãoAcad_SatEstudos'] = df_test_fe['Pressão Acadêmica'] *
    ↪df_test_fe['Satisfação com os Estudos']
df_train_fe['PressãoTrb_SatTrb'] = df_train_fe['Pressão no Trabalho'] *
    ↪df_train_fe['Satisfação com o Trabalho']
df_test_fe['PressãoTrb_SatTrb'] = df_test_fe['Pressão no Trabalho'] *
    ↪df_test_fe['Satisfação com o Trabalho']
df_train_fe['HorasIdade'] = df_train_fe['Horas de Trabalho/Estudo'] *
    ↪df_train_fe['Idade']
df_test_fe['HorasIdade'] = df_test_fe['Horas de Trabalho/Estudo'] *
    ↪df_test_fe['Idade']

# Binarizações
age_bins = [0, 25, 35, 45, 55, 200]
age_labels = [0, 1, 2, 3, 4]
df_train_fe['Idade_bin'] = pd.cut(df_train_fe['Idade'], bins=age_bins,
    ↪labels=age_labels, right=False).astype(int)
df_test_fe['Idade_bin'] = pd.cut(df_test_fe['Idade'], bins=age_bins,
    ↪labels=age_labels, right=False).astype(int)
df_train_fe['HighWorkload'] = (df_train_fe['Horas de Trabalho/Estudo'] > 8).
    ↪astype(int)
df_test_fe['HighWorkload'] = (df_test_fe['Horas de Trabalho/Estudo'] > 8).
    ↪astype(int)

# Aggregations
df_train_fe['Nome_word_count'] = df_train_fe['Nome'].str.split().apply(len)
df_test_fe['Nome_word_count'] = df_test_fe['Nome'].str.split().apply(len)
city_pressure_mean = df_train_fe.groupby('Cidade')['Pressão Acadêmica'].
    ↪transform('mean')
df_train_fe['CityPressMean'] = city_pressure_mean

```

```

city_pressure_mean_test = df_test_fe['Cidade'].map(df_train_fe.
    ↳groupby('Cidade')['Pressão Acadêmica'].mean())
df_test_fe['CityPressMean'] = city_pressure_mean_test
global_press_mean = df_train_fe['Pressão Acadêmica'].mean()
df_test_fe['CityPressMean'].fillna(global_press_mean, inplace=True)

# Transformations
df_train_fe['log_PressãoAcad'] = np.log1p(df_train_fe['Pressão Acadêmica'])
df_test_fe['log_PressãoAcad'] = np.log1p(df_test_fe['Pressão Acadêmica'])
df_train_fe['sqrt_Horas'] = np.sqrt(df_train_fe['Horas de Trabalho/Estudo'])
df_test_fe['sqrt_Horas'] = np.sqrt(df_test_fe['Horas de Trabalho/Estudo'])

# New feature list
new_feature_cols = [
    'PressãoAcad_SatEstudos',
    'PressãoTrb_SatTrb',
    'HorasIdade',
    'Idade_bin',
    'HighWorkload',
    'Nome_word_count',
    'CityPressMean',
    'log_PressãoAcad',
    'sqrt_Horas'
]

# 8. Transform original features
X_train = preprocessor.fit_transform(X_train_raw)
X_test = preprocessor.transform(X_test_raw)

# 9. Concatenate new features
train_new_feats = df_train_fe[new_feature_cols].values
test_new_feats = df_test_fe[new_feature_cols].values
X_train_fe = np.hstack([X_train, train_new_feats])
X_test_fe = np.hstack([X_test, test_new_feats])

# 10. Train/validation split
X_train_split, X_val_split, y_train_split, y_val_split = train_test_split(
    X_train_fe, y_train, test_size=0.20, random_state=42, stratify=y_train
)

# 11. Hyper-parameter search (lightweight)
xgb_base = XGBClassifier(
    n_jobs=4,
    random_state=42,
    objective='binary:logistic',
    eval_metric='logloss',
    use_label_encoder=False,

```

```

        verbosity=0,
        tree_method='hist'
    )

param_distributions = {
    'n_estimators': [100, 150, 200],
    'max_depth': [3, 4, 5, 6],
    'learning_rate': [0.05, 0.1, 0.2],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0],
    'gamma': [0, 0.1],
    'min_child_weight': [1, 3],
    'reg_alpha': [0, 0.01],
    'reg_lambda': [1, 1.5]
}

random_search = RandomizedSearchCV(
    estimator=xgb_base,
    param_distributions=param_distributions,
    n_iter=10,
    scoring='accuracy',
    cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=42),
    verbose=0,
    n_jobs=1,
    random_state=42
)

random_search.fit(X_train_split, y_train_split)

print('Melhores parâmetros encontrados:')
print(random_search.best_params_)
print(f"Melhor acurácia (CV): {random_search.best_score_:.4f}")

# 12. Evaluate on validation set
best_xgb = random_search.best_estimator_
val_pred = best_xgb.predict(X_val_split)
val_acc = accuracy_score(y_val_split, val_pred)
print(f'Acurácia do melhor modelo no conjunto de validação: {val_acc:.4f}')

# 13. Predict on test set and create submission
test_pred = best_xgb.predict(X_test_fe)
submission = pd.DataFrame({'id': df_test['id'], 'Depressão': test_pred.
    ↪astype(int)})
submission.to_csv('submission.csv', index=False)
print('Arquivo submission.csv criado com sucesso!')

```

Melhores parâmetros encontrados:

```
{'subsample': 0.8, 'reg_lambda': 1.5, 'reg_alpha': 0, 'n_estimators': 150,
```

```
'min_child_weight': 3, 'max_depth': 4, 'learning_rate': 0.2, 'gamma': 0,
'colsample_bytree': 0.8}
```

Melhor acurácia (CV): 0.9395

Acurácia do melhor modelo no conjunto de validação: 0.9386

Arquivo submission.csv criado com sucesso!

```
/tmp/ipykernel_91300/3080547404.py:81: FutureWarning: A value is trying to be
set on a copy of a DataFrame or Series through chained assignment using an
inplace method.
```

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df_test_fe['CityPressMean'].fillna(global_press_mean, inplace=True)
/home/jao/Mestrado/Projeto/Mestrado-Privado/p3_11-venv/lib/python3.11/site-
packages/xgboost/sklearn.py:1421: UserWarning: `use_label_encoder` is deprecated
in 1.7.0.
```

```
warnings.warn("`use_label_encoder` is deprecated in 1.7.0.")
/home/jao/Mestrado/Projeto/Mestrado-Privado/p3_11-venv/lib/python3.11/site-
packages/xgboost/sklearn.py:1421: UserWarning: `use_label_encoder` is deprecated
in 1.7.0.
```

```
warnings.warn("`use_label_encoder` is deprecated in 1.7.0.")
/home/jao/Mestrado/Projeto/Mestrado-Privado/p3_11-venv/lib/python3.11/site-
packages/xgboost/sklearn.py:1421: UserWarning: `use_label_encoder` is deprecated
in 1.7.0.
```

```
warnings.warn("`use_label_encoder` is deprecated in 1.7.0.")
/home/jao/Mestrado/Projeto/Mestrado-Privado/p3_11-venv/lib/python3.11/site-
packages/xgboost/sklearn.py:1421: UserWarning: `use_label_encoder` is deprecated
in 1.7.0.
```

```
warnings.warn("`use_label_encoder` is deprecated in 1.7.0.")
/home/jao/Mestrado/Projeto/Mestrado-Privado/p3_11-venv/lib/python3.11/site-
packages/xgboost/sklearn.py:1421: UserWarning: `use_label_encoder` is deprecated
in 1.7.0.
```

```
warnings.warn("`use_label_encoder` is deprecated in 1.7.0.")
/home/jao/Mestrado/Projeto/Mestrado-Privado/p3_11-venv/lib/python3.11/site-
packages/xgboost/sklearn.py:1421: UserWarning: `use_label_encoder` is deprecated
in 1.7.0.
```

```
warnings.warn("`use_label_encoder` is deprecated in 1.7.0.")
/home/jao/Mestrado/Projeto/Mestrado-Privado/p3_11-venv/lib/python3.11/site-
packages/xgboost/sklearn.py:1421: UserWarning: `use_label_encoder` is deprecated
in 1.7.0.
```

```
warnings.warn("`use_label_encoder` is deprecated in 1.7.0.")
/home/jao/Mestrado/Projeto/Mestrado-Privado/p3_11-venv/lib/python3.11/site-
```



```
packages/xgboost/sklearn.py:1421: UserWarning: `use_label_encoder` is deprecated in 1.7.0.
```

```
warnings.warn("`use_label_encoder` is deprecated in 1.7.0.")
```

1.9 Etapa 9 – Treinamento do Modelo Final com os Melhores Hiperparâmetros

1.9.1 Visão geral do que já foi feito

- **Carregamento e EDA:** os arquivos `train.csv` e `test.csv` foram lidos e analisados, identificando tipos de dados, valores ausentes e distribuição da variável alvo.
- **Pré-processamento:** imputação, codificação (Ordinal + OneHot) e normalização numérica foram aplicadas, gerando `X_train_fe` ($140\,700 \times 778$) e `X_test_fe` ($93\,800 \times 778$).
- **Engenharia de Features:** nove features adicionais foram criadas e concatenadas ao pipeline de pré-processamento.
- **Divisão em treino/validação:** 80/20 com `stratify=y_train` para avaliação do modelo base.
- **Modelo Base:** `XGBClassifier` com parâmetros padrão obteve acurácia de **0.9388** no conjunto de validação.
- **Otimização de Hiperparâmetros:** `RandomizedSearchCV` (3 folds, 10 iterações) encontrou os melhores parâmetros:

```
{
    'subsample': 0.8,
    'reg_lambda': 1.5,
    'reg_alpha': 0,
    'n_estimators': 150,
    'min_child_weight': 3,
    'max_depth': 4,
    'learning_rate': 0.2,
    'gamma': 0,
    'colsample_bytree': 0.8
}
```

A acurácia média de validação cruzada foi **0.9395**, e a acurácia no conjunto de validação hold-out foi **0.9386**.

1.9.2 Próximo passo – Treinar no conjunto completo

Para garantir que o modelo final aproveite todos os dados disponíveis, vamos:

1. **Re-instanciar** o `XGBClassifier` com os parâmetros otimizados e `n_jobs=16` (para usar todos os núcleos disponíveis).
2. **Fit** o modelo em **todas** as linhas de `X_train_fe` e `y_train`.
3. **Prever** a classe binária para cada linha de `X_test_fe`.
4. **Criar** o arquivo `submission.csv` com as colunas `id`, `Depressão`, usando os `id` originais de `df_test`.

1.9.3 Observações

- O pipeline de pré-processamento já foi ajustado no passo anterior, portanto não precisamos refazê-lo.

- Garantimos que o modelo final seja treinado apenas no conjunto de treino completo, evitando vazamento de informação.
- O arquivo `submission.csv` será sobrescrito, mantendo a estrutura exigida pelo Kaggle.
- Caso haja algum erro de memória ou tempo, podemos reduzir `n_estimators` ou usar `tree_method='hist'` para acelerar.

Com isso, concluímos a etapa de treinamento final e geraremos a submissão que será enviada ao Kaggle.

```
[3]: from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

# Instanciar o classificador com os melhores hiperparâmetros
best_params = {
    'subsample': 0.8,
    'reg_lambda': 1.5,
    'reg_alpha': 0,
    'n_estimators': 150,
    'min_child_weight': 3,
    'max_depth': 4,
    'learning_rate': 0.2,
    'gamma': 0,
    'colsample_bytree': 0.8
}

final_xgb = XGBClassifier(
    objective='binary:logistic',
    eval_metric='logloss',
    use_label_encoder=False,
    verbosity=0,
    tree_method='hist',
    n_jobs=16,
    random_state=42,
    **best_params
)

# Treinar no conjunto completo
final_xgb.fit(X_train_fe, y_train)

# Avaliar (opcional) no conjunto de validação hold-out
val_pred = final_xgb.predict(X_val_split)
val_acc = accuracy_score(y_val_split, val_pred)
print(f'Acurácia no conjunto de validação (hold-out): {val_acc:.4f}')

# Previsões para o conjunto de teste
test_pred = final_xgb.predict(X_test_fe)

# Criar DataFrame de submissão
```

```

submission = pd.DataFrame({
    'id': test_ids,
    'Depressão': test_pred.astype(int)
})

# Salvar em submission.csv
submission.to_csv('submission.csv', index=False)
print('Arquivo submission.csv criado com sucesso!')

```

Acurácia no conjunto de validação (hold-out): 0.9427

/home/jao/Mestrado/Projeto/Mestrado-Privado/p3_11-venv/lib/python3.11/site-packages/xgboost/sklearn.py:1421: UserWarning: `use\_label\_encoder` is deprecated in 1.7.0.

```
warnings.warn("`use_label_encoder` is deprecated in 1.7.0.")
```

```

-----
NameError                                Traceback (most recent call last)
Cell In[3], line 41
    37 test_pred = final_xgb.predict(X_test_fe)
    39 # Criar DataFrame de submissão
    40 submission = pd.DataFrame({
---> 41     'id': test_ids,
    42     'Depressão': test_pred.astype(int)
    43 })
    45 # Salvar em submission.csv
    46 submission.to_csv('submission.csv', index=False)

NameError: name 'test_ids' is not defined

```

```

[4]: from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score
import pandas as pd

# Instanciar o classificador com os melhores hiperparâmetros
best_params = {
    'subsample': 0.8,
    'reg_lambda': 1.5,
    'reg_alpha': 0,
    'n_estimators': 150,
    'min_child_weight': 3,
    'max_depth': 4,
    'learning_rate': 0.2,
    'gamma': 0,
    'colsample_bytree': 0.8
}

```

```

final_xgb = XGBClassifier(
    objective='binary:logistic',
    eval_metric='logloss',
    use_label_encoder=False,
    verbosity=0,
    tree_method='hist',
    n_jobs=16,
    random_state=42,
    **best_params
)

# Treinar no conjunto completo
final_xgb.fit(X_train_fe, y_train)

# Avaliar (opcional) no conjunto de validação hold-out
val_pred = final_xgb.predict(X_val_split)
val_acc = accuracy_score(y_val_split, val_pred)
print(f'Acurácia no conjunto de validação (hold-out): {val_acc:.4f}')

# Previsões para o conjunto de teste
test_pred = final_xgb.predict(X_test_fe)

# Criar DataFrame de submissão
submission = pd.DataFrame({
    'id': df_test['id'],
    'Depressão': test_pred.astype(int)
})

# Salvar em submission.csv
submission.to_csv('submission.csv', index=False)
print('Arquivo submission.csv criado com sucesso!')

```

Acurácia no conjunto de validação (hold-out): 0.9427

Arquivo submission.csv criado com sucesso!

/home/jao/Mestrado/Projeto/Mestrado-Privado/p3_11-venv/lib/python3.11/site-packages/xgboost/sklearn.py:1421: UserWarning: `use\_label\_encoder` is deprecated in 1.7.0.

warnings.warn("`use\_label\_encoder` is deprecated in 1.7.0.")

1.10 Etapa 10 – Geração da Submissão Final

1.10.1 Visão geral do que já foi feito

- **Carregamento e EDA:** os arquivos `train.csv` e `test.csv` foram lidos, tipos de dados analisados e valores ausentes tratados.
- **Pré-processamento:** imputação, codificação (Ordinal + OneHot) e normalização numérica foram aplicadas, resultando em `X_train_fe` ($140\,700 \times 778$) e `X_test_fe` ($93\,800 \times 778$).
- **Engenharia de Features:** nove features adicionais foram criadas e concatenadas ao pipeline

de pré-processamento.

- **Divisão em treino/validação:** 80/20 com `stratify=y_train` para avaliação do modelo.
- **Otimização de Hiperparâmetros:** `RandomizedSearchCV` encontrou os melhores parâmetros, que foram usados para treinar o modelo final.
- **Treinamento Final:** o `XGBClassifier` foi treinado no conjunto completo de treino, alcançando **acurácia de 0.9427** no conjunto hold-out.

1.10.2 O que a célula atual faz

1. **Re-instancia** o classificador com os hiperparâmetros otimizados e `n_jobs=16` para aproveitar todos os núcleos disponíveis.
2. **Fit** o modelo em todas as linhas de `X_train_fe` e `y_train`.
3. **Previsões** são geradas para o conjunto de teste (`X_test_fe`).
4. Um `DataFrame` com as colunas `id` e `Depressão` é criado e salvo em `submission.csv`.
5. O arquivo `submission.csv` está pronto para ser enviado ao Kaggle.

1.10.3 Próximos passos

- Se desejar experimentar outras abordagens (por exemplo, `CatBoost`, `LightGBM` ou modelos de ensemble), basta repetir o processo de pré-processamento e treinar o novo modelo.
- Para enviar a submissão, basta fazer upload do arquivo `submission.csv` na plataforma Kaggle.
- Caso queira re-executar a etapa de treinamento final com ajustes nos hiperparâmetros, altere o dicionário `best_params` e rode a célula novamente.

Parabéns! O pipeline completo está concluído e a submissão final foi gerada com sucesso.

```
[5]: import pandas as pd

# Gerar previsões finais para o conjunto de teste
test_pred = final_xgb.predict(X_test_fe)

# Criar DataFrame de submissão
submission = pd.DataFrame({
    'id': df_test['id'],
    'Depressão': test_pred.astype(int)
})

# Salvar em submission.csv
submission.to_csv('submission.csv', index=False)
print('Arquivo submission.csv criado com sucesso!')
```

Arquivo `submission.csv` criado com sucesso!